Teach Me Skills

Continuous Integration.

CircleCI u Jenkins

Continuous Integration

Непрерывная интеграция (CI, англ. Continuous Integration) — практика разработки программного обеспечения, которая заключается в постоянном слиянии рабочих копий в общую основную ветвь разработки (до нескольких раз в день) и выполнении частых автоматизированных сборок проекта для скорейшего выявления потенциальных дефектов и решения интеграционных проблем. Пример:

1. Сборка продукта
2. Запуск юнит-тестов
3. Запуск integration/UI тестов
4. Генерация репорта
5. Отсылка результатов в мессенджере/имейле

teachmeskills.by

Способы запуска

1. Вручную
2. Запуск каждые N дней/часов/минут
3. Запуск после каждого коммита/PR

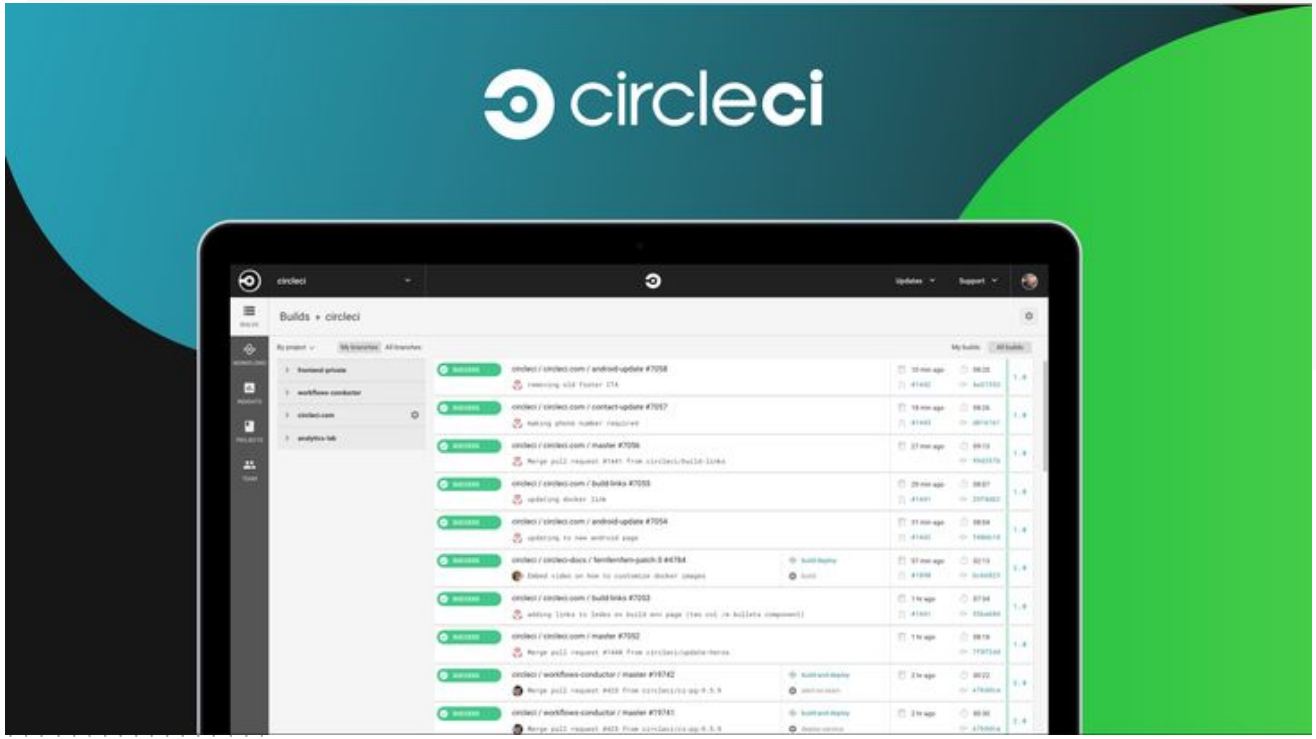
teachmeskills.by

Основные инструменты CI

- https://jenkins.io - установка локально либо на сервер, бесплатное использование, но придется искать хостинг с доступом к файловой системе
- https://travis-ci.org - работает в облаке, бесплатный для продуктов с открытым кодом
- https://circleci.com - работает в облаке, бесплатный 2500 минут в неделю
- https://www.jetbrains.com/teamcity/ - работает в облаке, платный

teachmeskills.by

CircleCI



teachmeskills.by

CircleCI - настройка

1. Заходим на https://circleci.com/vcs-authorize/
2. Привязываем свой GitHub-аккаунт
3. Выбираем проект для которого хотим настроить CI
4. Далее - Далее - Готово
5. …
6. Ничерта не работает

teachmeskills.by

```
$ #!/bin/sh -eo pipefail                                                    Exit code: 1
  # No configuration was found in your project. Please refer to https://circleci.com/docs/2.0/ to get started with your configuration.
  #
  # -------
  # Warning: This configuration was auto-generated to show you the message above.
  # Don't rerun this job. Rerunning will have no effect.
  false

Exited with code 1
```

Виртуалки и докеры

Виртуальная машина (VM, от англ. virtual machine) — программная и/или аппаратная система, эмулирующая аппаратное обеспечение некоторой платформы (target — целевая, или гостевая платформа) и исполняющая программы для target-платформы на host-платформе (host — хост-платформа, платформа-хозяин) или виртуализирующая некоторую платформу и создающая на ней среды, изолирующие друг от друга программы и даже операционные системы (см.: песочница);

Docker — программное обеспечение для автоматизации развёртывания и управления приложениями в средах с поддержкой контейнеризации. Позволяет «упаковать» приложение со всем его окружением и зависимостями в контейнер, который может быть перенесён на любую Linux-систему с поддержкой cgroups в ядре, а также предоставляет среду по управлению контейнерами.

teachmeskills.by

CircleCI - настройка

1. В своем проекте в IDE создаем папку .circleci
2. В нем создаем файл config.yml
3. Добавляем туда следующий контент:

   https://github.com/dzmitryrak/AllureReporting/blob/master/.circleci/config.yml
4. Коммитим
5. Далее - Далее - Готово
6. …
7. Ничерта не работает

teachmeskills.by

```
SEVERE: org.apache.commons.exec.ExecuteException: Process exited with an error: 2 (Exit value: 2)
[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 2, Time elapsed: 21.535 s <<< FAILURE! - in tests.GoogleTest
[ERROR] createDriver(tests.GoogleTest)  Time elapsed: 21.439 s  <<< FAILURE!
org.openqa.selenium.WebDriverException:
Timed out waiting for driver server to start.
Build info: version: '3.141.59', revision: 'e82be7d358', time: '2018-11-14T08:17:03'
System info: host: '66c9815ee0a6', ip: '172.25.0.3', os.name: 'Linux', os.arch: 'amd64', os.version: '4.15.0-1043-aws', java.version: '1.8.0_181'
Driver info: driver.version: ChromeDriver
Caused by: org.openqa.selenium.net.UrlChecker$TimeoutException: Timed out waiting for [http://localhost:27275/status] to be available after 20000 ms
Caused by: java.util.concurrent.TimeoutException
```

Окружение

1. Выбор операционной системы
2. Выбор железа

teachmeskills.by

Возможные проблемы

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.0.0-M3</version>
  <configuration>
     <forkCount>2</forkCount>
     <reuseForks>true</reuseForks>
     <argLine>

-javaagent:${settings.localRepository}/org/aspectj/aspectjweaver/1.9.1/aspectjweaver-1.9.1.jar
-Xmx256m
     </argLine>
     <useSystemClassLoader>false</useSystemClassLoader>
  </configuration>
  <dependencies>
     <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjweaver</artifactId>
        <version>1.9.1</version>
     </dependency>
  </dependencies>
</plugin>
```

teachmeskills.by

Настройка

1

Репортинг:

mvn allure:report

- store_artifacts: path: target/site/allure-maven-plugin/

2

Настройка запуска:

```
workflows:
 version: 2
 commit:
   jobs:
     - ui-tests
 periodical:
   triggers:
     - schedule:
         cron: "0 8 * * *"
         filters:
           branches:
             only:
               - master
   jobs:
     - ui-tests
```

teachmeskills.by

А если тесты падают?

1. Скриншоты-скриншоты-скриншоты: https://github.com/dzmitryrak/AllureReporting/blob/master/src/test/java/tests/base/TestListener.java

Прячем секретные данные

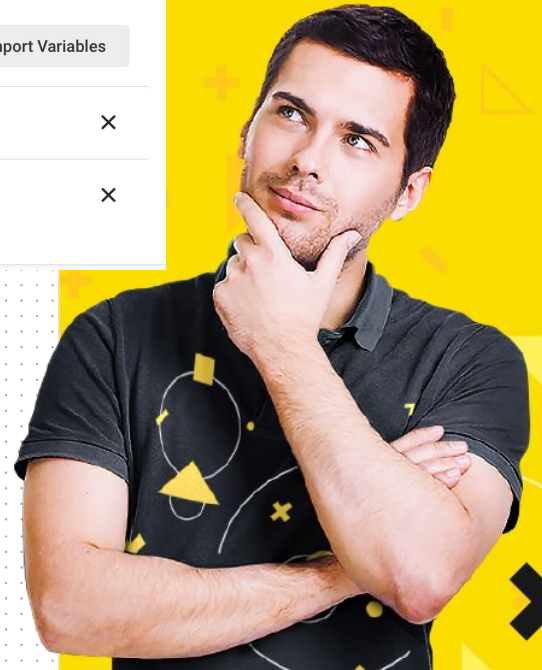
Environment Variables

Environment variables let you add sensitive data (e.g. API keys) to your jobs rather than placing them in the repository. The value of the variables cannot be read or edited in the app once they are set.

If you're looking to share environment variables across projects, try **Contexts**.

Name	Value	Add Variable	Import Variables
password	xxxxword		×
user	xxxxrahk		×

```
public void printSensitiveData() {
  // gets the value of the specified environment variable "PATH"
  System.out.println(System.getenv("PATH"));

  // gets the value of the specified environment variable "USERNAME"
  System.out.println(System.getenv("USERNAME"));

  // gets the value of the specified environment variable "PASSWORD"
  System.out.println(System.getenv("PASSWORD"));
}
```



teachmeskills.by

Чтение через PropertyReader

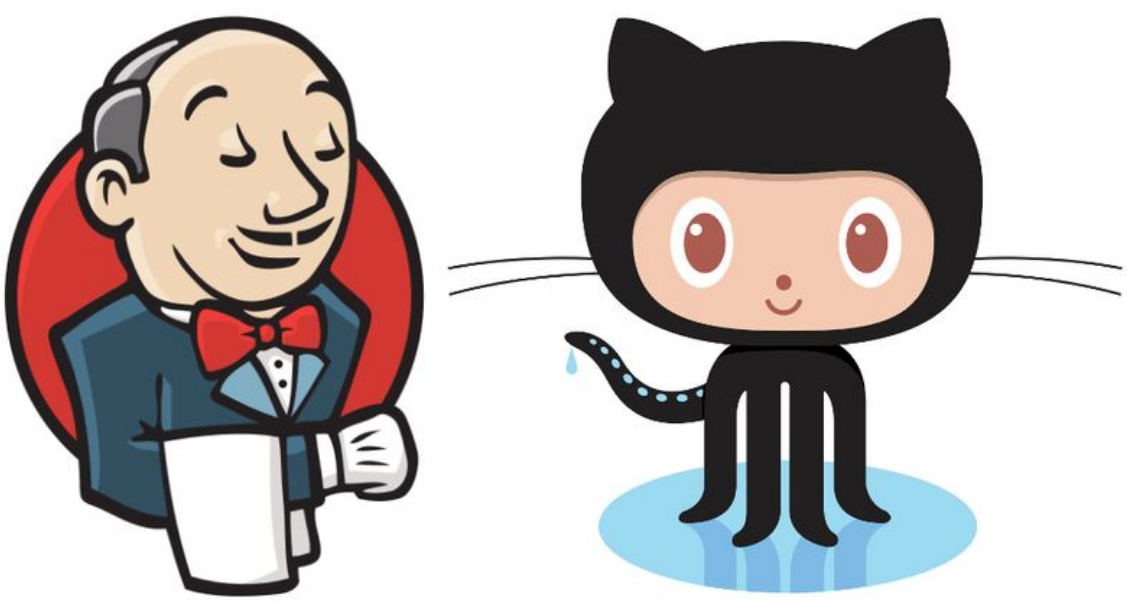
- Пример реализации статического PropertyManager: https://github.com/dzmitryrak/AllureReporting/blob/master/src/test/java/utils/PropertyReader.java
- Возможно использование в связке с System.getEnv()

```
System.out.println(System.getenv().getOrDefault("PATH",
PropertyReader.getProperty("PATH")));
```

teachmeskills.by

Jenkins + Github

Continuous Integration (Jenkins, Git) - позволяют запускать тесты не на вашем компьютере, а на удаленном сервере и по определенным правилам/расписанию. Кроме того вся команда имеет доступ к коду



teachmeskills.by

Действия

1. Установка
2. Конфигурация
3. Запуск
4. Создание job/pipeline

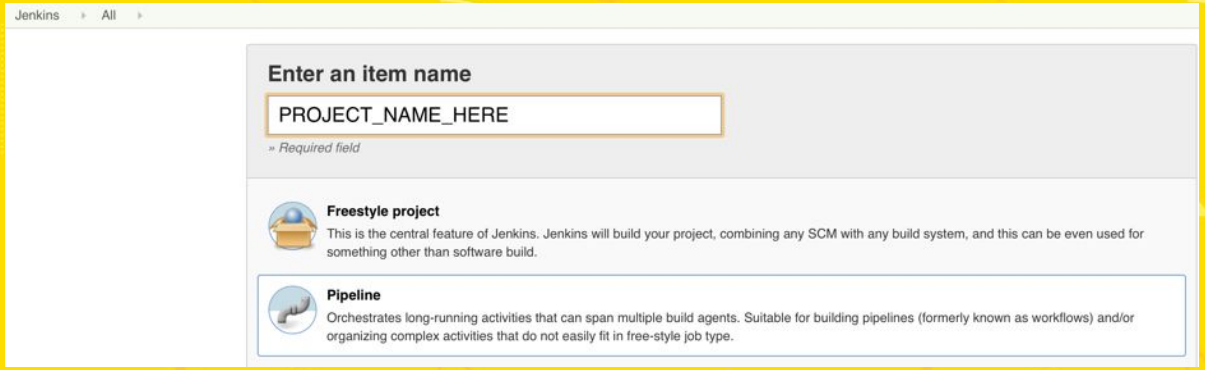
teachmeskills.by

Установка и запуск Jenkins

1. Установка:
   a. Mac: https://jenkins.io/download/lts/macos/
   b. Windows: https://jenkins.io/download/thank-you-downloading-windows-installer-stable/
2. Кофигурация:
   a. Mac: https://jenkins.io/download/lts/macos/
   b. Windows: https://jenkins.io/doc/book/installing/#windows
3. Открыть http://localhost:8080

teachmeskills.by

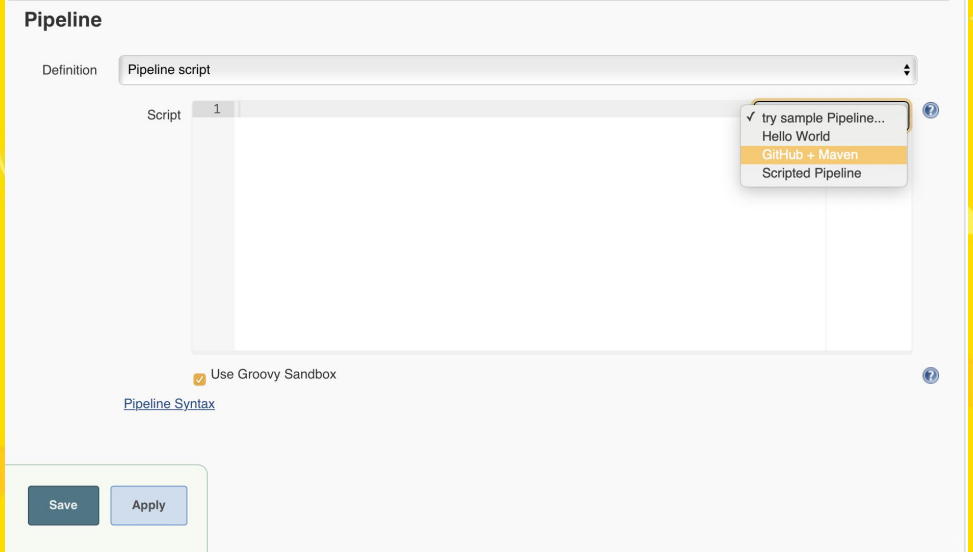
Создание Jenkins pipeline



https://jenkins.io/pipeline/getting-started-pipelines/

teachmeskills.by

Создание Jenkins pipeline



https://jenkins.io/doc/book/pipeline/jenkinsfile/

teachmeskills.by

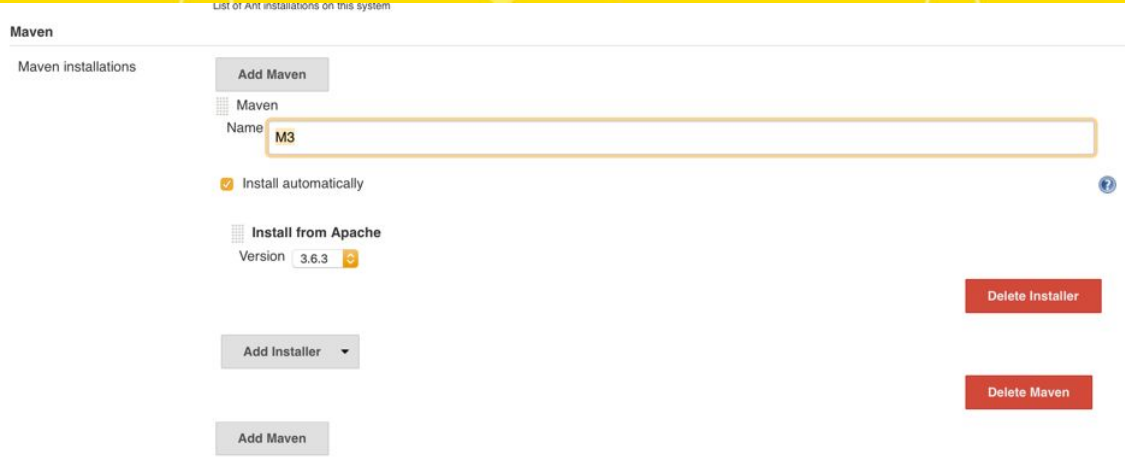
Jenkinsfile

```
pipeline {
  agent any

  tools {
    // Install the Maven version configured as "M3" and add it to the path.
    maven "M3"
  }

  stages {
    stage('Build') {
      steps {
        // Get some code from a GitHub repository
        git 'https://github.com/jglick/simple-maven-project-with-tests.git'

        // Run Maven on a Unix agent.
        sh "mvn -Dmaven.test.failure.ignore=true clean package"

        // To run Maven on a Windows agent, use
        // bat "mvn -Dmaven.test.failure.ignore=true clean package"
      }

      post {
        // If Maven was able to run the tests, even if some of the test
        // failed, record the test results and archive the jar file.
        success {
          junit '**/target/surefire-reports/TEST-*.xml'
          archiveArtifacts 'target/*.jar'
        }
      }
    }
  }
}
```

teachmeskills.by

Конфигурация Maven

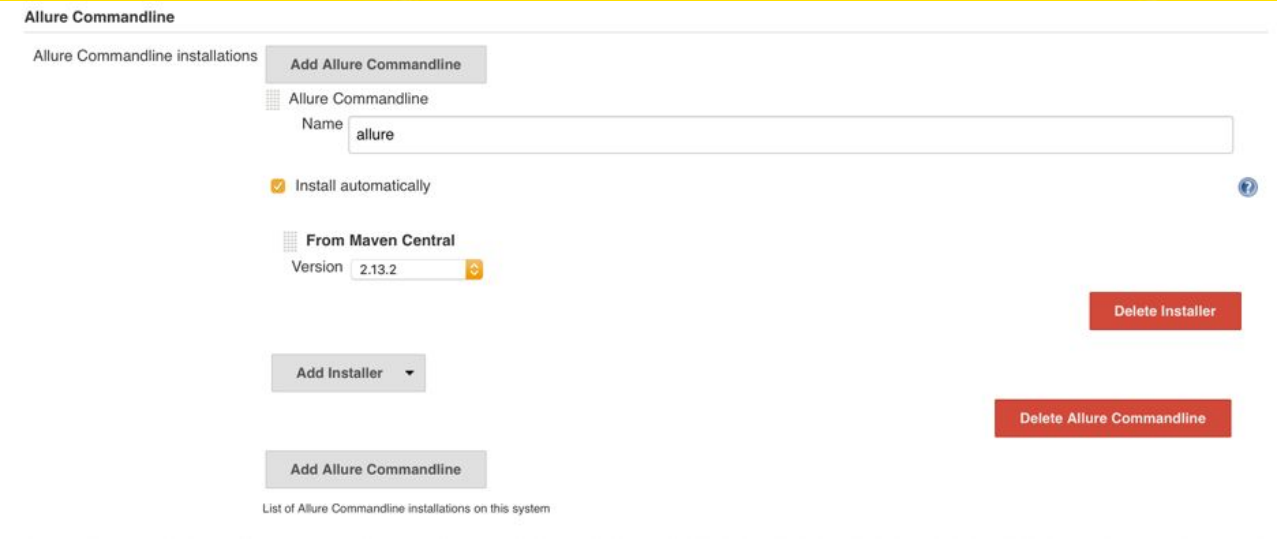
1. Идем в http://localhost:8080/configureTools/
2. Конфигурируем Tool под названием M3



teachmeskills.by

Конфигурация Allure

1. Устанавливаем Allure Plugin http://localhost:8080/pluginManager/
2. Конфигурируем Allure Commandline



teachmeskills.by

Allure stage в Jenkinsfile

```
stage('reports') {
 steps {
    script {
            allure([
                    includeProperties: false,
                    jdk: '',
                    properties: [],
                    reportBuildPolicy: 'ALWAYS',
                    results: [[path: 'target/allure-results']]
            ])
    }
 }
}
```

teachmeskills.by

Конфигурация GIT параметров

1. Устанавливаем Git Parameter Plugin http://localhost:8080/pluginManager/
2. Добавляем параметр для бранча в pipeline

```
tools {
   // Install the Maven version configured as "M3" and add it to the path.
   maven "M3"
}
parameters {
 gitParameter branchFilter: 'origin/(.*)', defaultValue: 'master', name: 'BRANCH', type:
'PT_BRANCH'
}


stages {
  stage('Run test') {
     steps {
        // Get some code from a GitHub repository
        git branch: "${params.BRANCH}", url: 'https://github.com/dzmitryrak/AllureReporting.git'
     }
  }
}
```

teachmeskills.by